architecture.py

`src/moe_pruning/model/architecture.py · 219 lines · module moe_pruning.model.architecture`

What it does

The model-independent interface between the pruning code and any MoE model. Defines what a sparse MoE layer is (`MoELayerInfo`), what one observed forward pass contains (`MoEForwardRecord`) and the operations every model adapter must provide (`MoEAdapter`).

How it works

- `MoELayerInfo`: layer id, stack, module paths, number of experts, top-k, dimensions and expert capacity.
- `MoEForwardRecord`: residual stream, MoE input and output, top-k expert indices and gates, and un-gated expert outputs for every token. All scoring and initialisation is computed from these records.
- `MoEAdapter`: discover layers, access router and experts, route (with or without a pruning mask), run one layer, instrument a forward pass, replace experts, check that removed weights are gone, load an original expert from disk.
- Default implementations cover models with one module per expert; fused-expert models override them.
- `select_layers()` resolves `PRUNE_LAYERS` (`first`, `all` or explicit ids). `get_adapter()` finds the adapter through the model plugin registry.

Notes

- Expert ids always refer to the original router indices, also after pruning.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class MoELayerInfo`	Static description of one sparse MoE layer.
`.to_dict()`	Convert to a dictionary (stored in manifests and reports).
`class MoEForwardRecord`	Everything observed in one forward call of one sparse MoE layer.
`class MoEAdapter`	Model-specific access to sparse MoE layers.
`.discover_layers(model)`	Find all sparse MoE layers by inspecting the loaded model.
`.get_moe_module(model, layer)`	The sparse MoE module of a layer.
`.get_router(model, layer)`	The router module of a layer.
`.get_expert(model, layer, expert_id)`	Module computing one expert (original, view or replacement).
`.set_expert(model, layer, expert_id, module)`	Install `module` in the expert slot and return the previous module.
`.build_original_expert(model, layer)`	New (uninitialised) expert module of the model's original expert class.
`.expert_state_prefix(layer, expert_id)`	State-dict key prefix of an expert slot (ends with a dot).
`.route(model, layer, moe_input, apply_mask)`	Return (expert_index [B,S,K], gate [B,S,K]) exactly as used by the layer.
`.set_routing_mask(model, layer, expert_ids)`	Make `expert_ids` unreachable: their tokens go to the best remaining expert.
`.get_routing_mask(model, layer)`	Experts currently made unreachable by the reroute mask.
`.run_moe_layer(model, layer, moe_input)`	Execute only the sparse MoE module on `moe_input` ([B,S,D]).
`.instrument(model, layers, callback)`	Call `callback` with a `MoEForwardRecord` per layer forward.
`.describe(model)`	Machine-readable architecture report.

Name	Description
`.replace_experts_in_layer(model, layer, modules)`	Install `modules` in their expert slots; return the number of removed parameters.
`.removed_weight_violations(state_shapes, layer, removed)`	Names of stored tensors that still hold weights of removed experts.
`.load_original_expert(reader, model, layer, expert_id)`	Rebuild one original expert from the original checkpoint (`reader`: CheckpointReader).
`.validate_loaded(model, layers)`	Called by the checkpoint loader after weights are loaded; raise on inconsistency.
`.get_experts(model, layer)`	All expert modules of a layer by id.
`.hidden_dims(model)`	Hidden and expert dimensions of every MoE layer.
`select_layers(layers, spec)`	Resolve PRUNE_LAYERS: `first`, `all` or comma-separated layer ids.
`get_adapter(model)`	Adapter of the model's plugin (looked up by `config.model_type`).

Connections

Uses: `moe_pruning.models`

Used by: `scripts/collect_stats.py`, `scripts/evaluate.py`, `scripts/export_model.py`, `scripts/inspect_model.py`, `scripts/replace_experts.py`, `scripts/score_experts.py`, `scripts/train_biases.py`, `src/moe_pruning/evaluation/evaluator.py`, `src/moe_pruning/model/instrumentation.py`, `src/moe_pruning/models/__init__.py`, `src/moe_pruning/models/olmoe/adapter.py`, `src/moe_pruning/models/switch_transformers/adapter.py`, `src/moe_pruning/pruning/expert_selection.py`, `src/moe_pruning/pruning/manifest.py`, `src/moe_pruning/pruning/replacement.py`, `src/moe_pruning/scoring/statistics.py`, `src/moe_pruning/serialization/checkpoint.py`, `src/moe_pruning/training/bias_trainer.py`